

IT3071 – Machine Learning and Optimization Methods

Activation Functions & Loss Functions

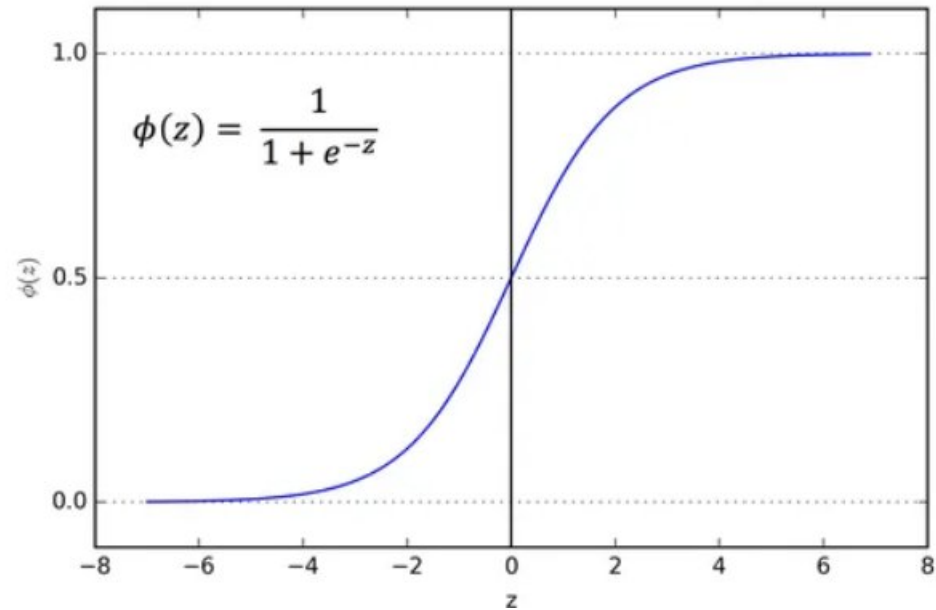
H.M. Samadhi Chathuranga Rathnayake

What is Activation Function?

- It's just a thing function that you use to get the output of node.
- It is also known as Transfer Function.
- It is used to determine the output of neural network like yes or no.
- It maps the resulting values in between 0 to 1 or -1 to 1 etc. (depending upon the function).

Sigmoid or Logistic Activation Function

- The **Sigmoid** Function curve looks like a S-shape.

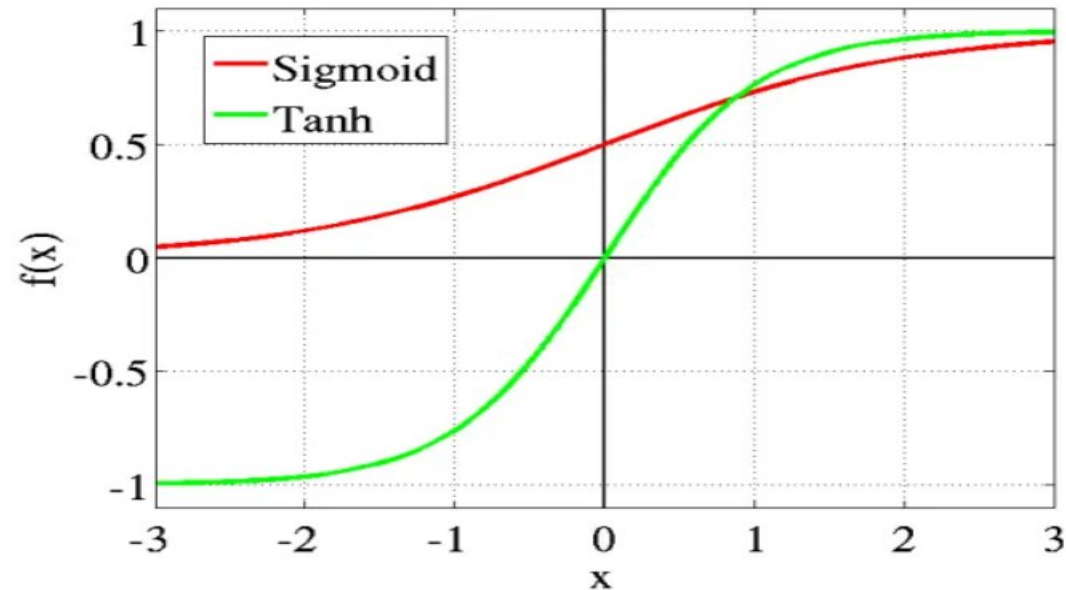


Sigmoid or Logistic Activation Function

- The main reason why we use sigmoid function is because it exists between (0 to 1).
- Therefore, it is especially used for models where we have to predict the probability as an output.
- Since probability of anything exists only between the range of 0 and 1, sigmoid is the right choice.
- The function is differentiable.
- That means, we can find the slope of the sigmoid curve at any two points.
- The function is monotonic, but function's derivative is not.
- The logistic sigmoid function can cause a neural network to get stuck at the training time.
- The softmax function is a more generalized logistic activation function which is used for multiclass classification.

Tanh or hyperbolic tangent Activation Function

- **tanh** is also like logistic sigmoid but better.
- The range of the tanh function is from (-1 to 1).
- tanh is also sigmoidal (s - shaped).

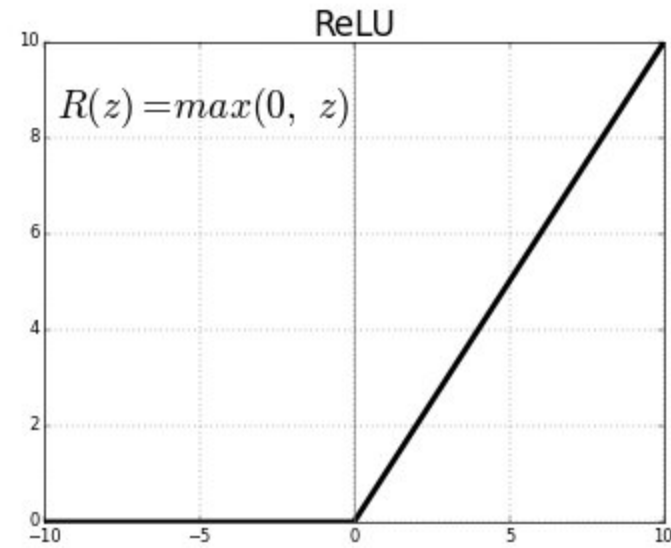
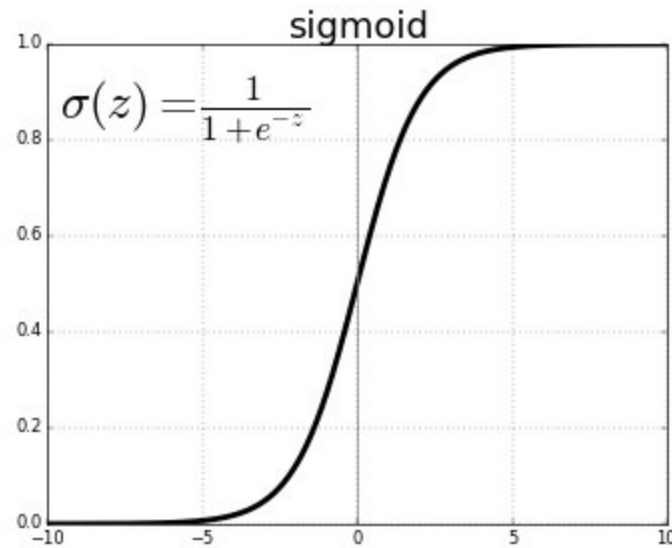


Tanh or hyperbolic tangent Activation Function

- The advantage is that the negative inputs will be mapped strongly negative, and the zero inputs will be mapped near zero in the tanh graph.
- The function is differentiable.
- The function is monotonic while its derivative is not monotonic.
- The tanh function is mainly used classification between two classes.
- Both tanh and logistic sigmoid activation functions are used in feed-forward nets.

ReLU (Rectified Linear Unit) Activation Function

- The **ReLU** is the most used activation function in the world right now.
- Since, it is used in almost all the convolutional neural networks or deep learning.



ReLU (Rectified Linear Unit) Activation Function

- As you can see, the ReLU is half rectified (from bottom). $f(z)$ is zero when z is less than zero and $f(z)$ is equal to z when z is above or equal to zero.
- Range: [0 to infinity)
- The function and its derivative both are monotonic.
- But the issue is that all the negative values become zero immediately which decreases the ability of the model to fit or train from the data properly.
- That means any negative input given to the ReLU activation function turns the value into zero immediately in the graph, which in turns affects the resulting graph by not mapping the negative values appropriately.

Softmax Function

- **Softmax** function is often described as a combination of multiple sigmoids.

$$\sigma(\mathbf{z})_j = \frac{e^{z_j}}{\sum_{k=1}^K e^{z_k}} \quad \text{for } j = 1, \dots, K.$$

Softmax Function

- We know that sigmoid returns values between 0 and 1, which can be treated as probabilities of a data point belonging to a particular class.
- Thus sigmoid is widely used for binary classification problems.
- The softmax function can be used for multiclass classification problems.
- This function returns the probability for a datapoint belonging to each individual class.

What is Loss Function?

- A loss function is a function that compares the target and predicted output values; measures how well the neural network models the training data.
- When training, we aim to minimize this loss between the predicted and target outputs.
- The hyperparameters are adjusted to minimize the average loss, we find the weights, w^T , and biases, b , that minimize the value of J (average loss).

$$J(w^T, b) = \frac{1}{m} \sum_{i=1}^m L(\hat{y}^{(i)}, y^{(i)})$$

Types of Loss Functions

- In supervised learning, there are two main types of loss functions.
- These correlate to the 2 major types of neural networks:
 - **Regression Loss Functions** — used in regression neural networks; given an input value, the model predicts a corresponding output value (rather than pre-selected labels); Ex. Mean Squared Error, Mean Absolute Error
 - **Classification Loss Functions** — used in classification neural networks; given an input, the neural network produces a vector of probabilities of the input belonging to various pre-set categories — can then select the category with the highest probability of belonging; Ex. Binary Cross-Entropy, Categorical Cross-Entropy

Mean Squared Error (MSE)

- One of the most popular loss functions, MSE finds the average of the squared differences between the target and the predicted outputs.

$$MSE = \frac{1}{n} \sum_{i=1}^n (y^{(i)} - \hat{y}^{(i)})^2$$

- This function has numerous properties that make it especially suited for calculating loss.
- The difference is squared, which means it does not matter whether the predicted value is above or below the target value; however, values with a large error are penalized.
- MSE is also a convex function (as shown in the diagram above) with a clearly defined global minimum, this allows us to more easily utilize gradient descent optimization to set the weight values.

Mean Squared Error (MSE)

- However, one disadvantage of this loss function is that it is very sensitive to outliers; if a predicted value is significantly greater than or less than its target value, this will significantly increase the loss.

Mean Absolute Error (MAE)

- MAE finds the average of the absolute differences between the target and the predicted outputs.

$$MAE = \frac{1}{n} \sum_{i=1}^n |y^{(i)} - \hat{y}^{(i)}|$$

- This loss function is used as an alternative to MSE in some cases.
- As mentioned previously, MSE is highly sensitive to outliers, which can dramatically affect the loss because the distance is squared.
- MAE is used in cases when the training data has a large number of outliers to mitigate this.

Binary Cross-Entropy/Log Loss

- This is the loss function used in binary classification models, where the model takes in an input and has to classify it into one of two pre-set categories.

$$CE\ Loss = \frac{1}{n} \sum_{i=1}^N - (y_i \cdot \log(p_i) + (1 - y_i) \cdot \log(1 - p_i))$$

- Classification neural networks work by outputting a vector of probabilities, the probability that the given input fits into each of the pre-set categories; then selecting the category with the highest probability as the final output.

Binary Cross-Entropy/Log Loss

- In binary classification, there are only two possible actual values of y — 0 or 1.
- Thus, to accurately determine loss between the actual and predicted values, it needs to compare the actual value (0 or 1) with the probability that the input aligns with that category ($p(i)$ = probability that the category is 1; $1 - p(i)$ = probability that the category is 0)

Categorical Cross-Entropy Loss

- In cases where the number of classes is greater than two, we utilize categorical cross-entropy , this follows a very similar process to binary cross-entropy.

$$CE\ Loss = -\frac{1}{n} \sum_{i=1}^N \sum_{j=1}^M y_{ij} \cdot \log(p_{ij})$$

- Binary cross-entropy is a special case of categorical cross-entropy, where $M = 2$, the number of categories is 2.

Custom Loss Functions

- Kullback-Leibler (KL) divergence loss
- Mean Absolute Error (MAE)
- Mean Absolute Percentage Error (MAPE)
- Mean Squared Error (MSE)
- Mean Squared Logarithmic Error (MSLE)
- Binary Crossentropy Loss
- Binary Focal Crossentropy Loss
- Sparse Categorical Crossentropy Loss
- Categorical Hinge Loss
- Hinge Loss
- Cosine Similarity
- Logcosh
- Huber loss
- Poisson loss